

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	T.Ashwath Ram
Register Number	3122247001007
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20.09.2026

1 Objective

To implement and compare the performance of two classifiers built from scratch on a multi-class handwritten-character recognition task:

- **Model A:** a Single-Layer Perceptron Learning Algorithm (PLA) using a step activation function, and
- **Model B:** a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, trained with back-propagation.

Hyperparameters for the MLP – activation function, cost function, optimizer, learning rate, number of hidden layers/neurons and batch size – are selected and justified through systematic tuning, and the two models are finally compared on a held-out test set using standard classification metrics.

2 Problem Statement

Given a 28×28 grayscale image of a handwritten character (flattened to a 784-dimensional feature vector), the task is to classify it into one of 62 classes (digits 0--9, uppercase A--Z, lowercase a--z). The **input** is the pixel-intensity vector of a single character image and the **output** is the predicted class label. The **objective** is to design, train and tune two families of classifiers – a linear single-layer perceptron and a non-linear multilayer perceptron – and to determine, through a controlled A/B comparison, which model generalises better to unseen handwritten characters and why.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (archive.zip – <code>Img/</code> folder of PNGs + <code>english.csv</code> label file)
Number of Samples	3,410 images (55 images per class)
Number of Features	784 (each image resized to 28×28 and flattened)
Number of Classes	62 (0--9, A--Z, a--z)
Missing Values	None – every listed image file is present and every class has exactly 55 samples

Train-Test Split	Stratified 70% train / 15% validation / 15% test $\Rightarrow$ 2,386 train, 512 validation, 512 test images (seed = 42)
------------------	---

4 Exploratory Data Analysis

The dataset consists of 1200×900 RGB photographs of a single handwritten character (black ink on a white background) per image, with exactly 55 samples for each of the 62 classes, so the raw class distribution is perfectly balanced. The key exploratory step here is inspecting the raw images and verifying that the preprocessing pipeline (Section 6) correctly isolates the character before it is fed to either model.

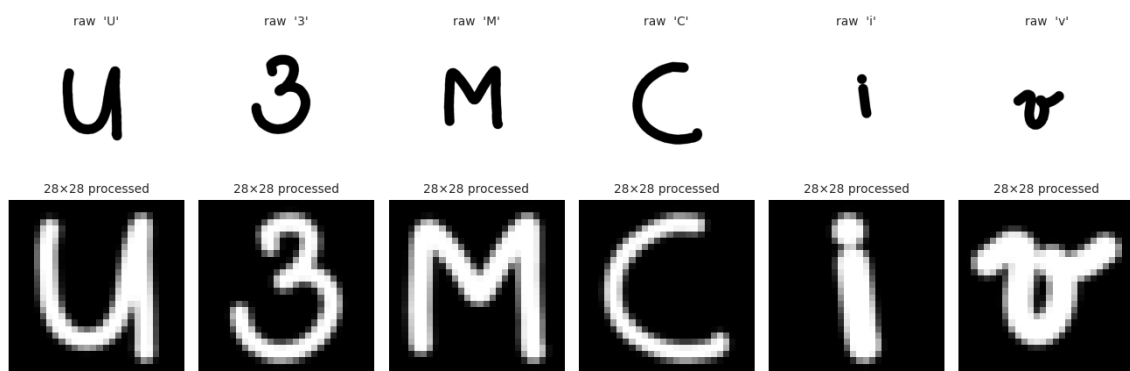


Figure 1: Six randomly sampled raw images (top row) and the corresponding 28×28 preprocessed versions (bottom row) used as model input.

Inference:

The raw photographs vary in stroke thickness, slant and position within the frame, which is exactly the kind of variability that makes this a non-trivial classification problem. After grayscale conversion, inversion (ink $\rightarrow$ high value), cropping to the character's bounding box and resizing to 28×28 , every character is centred and scaled consistently, which removes most of the position/scale variation while preserving the shape that separates one class from another. Because the class distribution is exactly balanced (55 images/class) and there are no missing values, the raw dataset needs no class-balancing or imputation step – the main preprocessing burden is purely geometric normalisation of the images themselves.

5 Data Preprocessing

- **Missing value handling:** Not required – every image referenced in `english.csv` exists and every class has the full complement of 55 samples.
- **Grayscale & inversion:** Each RGB image is converted to grayscale and inverted ($255 - \text{pixel}$) so that the ink strokes have high intensity and the background is 0, giving sparse, zero-background inputs that train more reliably.

- **Cropping & resizing:** Each image is cropped to the bounding box of the ink (threshold > 50), padded to a square with a 10% margin, and resized to 28×28 (784 features). This removes position/scale variation without distorting the character's aspect ratio.
- **Feature scaling:** Pixel values are scaled to $[0, 1]$ by dividing by 255, flattened into a 784-dimensional vector, and then standardised (zero mean, unit variance) using the mean/standard deviation computed *only* on the training split.
- **Encoding:** Class labels (characters) are label-encoded to integer indices 0–61, ordered as digits $\rightarrow$ uppercase $\rightarrow$ lowercase.
- **Train-test split:** A stratified 70/15/15 split (train/validation/test) preserves the balanced class proportions in every split; the validation set is used only for hyperparameter tuning and early stopping, and the test set is touched exactly once, at the very end.
- **Feature engineering:** Not applicable – raw (normalised, standardised) pixel intensities are used directly as the 784 input features for both models.

6 Machine Learning Algorithm

Model A: Single-Layer Perceptron (PLA)

Working principle. Each of the 62 output neurons computes a net input $z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$ and applies the step (Heaviside) activation $\hat{y}_k = \mathbb{I}[z_k \geq 0]$, so the problem is treated as 62 parallel one-vs-rest linear classifiers with one-hot targets. After every training sample the classical perceptron update rule is applied.

Mathematical formulation.

$$\mathbf{w}_k \leftarrow \mathbf{w}_k + \eta (y_k - \hat{y}_k) \mathbf{x}, \quad b_k \leftarrow b_k + \eta (y_k - \hat{y}_k)$$

At prediction time the class with the largest net input z_k (arg-max) is chosen, and the raw net inputs also serve as scores for the ROC curves.

Advantages: extremely simple, fast to train (a single epoch is $O(NDK)$), no hyperparameters beyond learning rate/epochs, and the update converges quickly on the training set.

Limitations: guaranteed convergence only if classes are linearly separable; handwritten characters are not, so the PLA can memorise the training set (train accuracy ≈ 0.98) while generalising poorly; the hard step output provides no gradient information and no calibrated probability scores.

Model B: Multilayer Perceptron (MLP)

Working principle. Architecture: Input (784) $\rightarrow$ one or more hidden layers with a non-linear activation $\rightarrow$ output layer (62 units, soft-max). The forward pass is $\mathbf{a}^{(l)} = f(\mathbf{a}^{(l-1)}W^{(l)} + \mathbf{b}^{(l)})$ with output $\mathbf{p} = \text{softmax}(\mathbf{z}^{(L)})$. Gradients are obtained via back-propagation, $\delta^{(L)} = \partial \mathcal{L} / \partial \mathbf{z}^{(L)}$ and $\delta^{(l)} = (\delta^{(l+1)}W^{(l+1)\top}) \odot f'(\mathbf{z}^{(l)})$, verified against a numerical (finite-difference) gradient check for every activation/loss pair before training (relative error $\ll 10^{-5}$).

Mathematical formulation. Supported cost functions: Cross-Entropy $-\frac{1}{N} \sum \sum y \log p$ and Mean-Squared-Error $\frac{1}{N} \sum \|\mathbf{p} - \mathbf{y}\|^2$; supported activations: ReLU, Tanh, Sigmoid (He initialisation for ReLU, Xavier otherwise); supported optimizers: mini-batch SGD, SGD + Momentum ($\beta = 0.9$), and Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$); L2 weight decay and early stopping (restoring the best validation-loss epoch) are available for over-fitting control.

Advantages: learns a non-linear hidden representation, so it can separate classes that are not linearly separable; the smooth soft-max/cross-entropy output gives calibrated class probabilities and a well-behaved gradient signal.

Limitations: many more hyperparameters to tune; more prone to over-fitting on a small dataset (only ≈ 38 training images per class here); more expensive to train than the PLA; poor hyperparameter choices (e.g. sigmoid + MSE) can make it perform *worse* than the simple PLA (observed: untuned MLP test accuracy = 0.143).

7 Experimental Setup

Python Version	3.x (NumPy / pandas / scikit-learn / Matplotlib / Seaborn / Pillow stack)
Scikit-Learn Version	Used only for metrics, splitting and label utilities (PLA and MLP are hand-coded)
IDE	Jupyter Notebook
Operating System	Linux
Hardware	CPU-only training (each MLP configuration trains in a few seconds)

8 Hyperparameter Selection

A full grid search over all MLP hyperparameters simultaneously would be prohibitively expensive, so a **stage-wise (coordinate) search** was used: one hyperparameter group is tuned at a time, always carrying forward the best value from the previous stage, with every configuration trained on the training set (early stopping on validation loss, max 60 epochs, patience 10) and scored by validation accuracy at the best epoch.

Stage	Hyperparameters searched	Values tried
1	Optimizer $\times$ Learning rate	SGD / Momentum / Adam $\times$ {0.001, 0.01, 0.1}
2	Activation $\times$ Cost function	ReLU / Tanh / Sigmoid $\times$ Cross-Entropy / MSE
3	Batch size	16, 32, 64, 128, 256
4	Hidden layers & neurons	(64,), (128,), (256,), (128,64), (256,128), (256,128,64), (128,128,128)
5	L2 regularisation	0, 10^{-4} , 10^{-3} , 10^{-2}

The starting (base) configuration was a single hidden layer of 128 neurons, ReLU activation, cross-entropy loss and batch size 64; 31 configurations were trained in total across the five stages.

Optimizer	LR	Best epoch	Train acc.	Val. acc.	Val. loss
Momentum	0.01	18	0.967	0.756	0.936
Adam	0.001	12	0.957	0.740	0.933
SGD	0.1	18	0.969	0.738	0.934
Adam	0.1	2	0.019	0.021	4.120

Table 3: Stage 1 (Optimizer $\times$ LR) – best and worst rows. Adam with a large learning rate diverges completely.

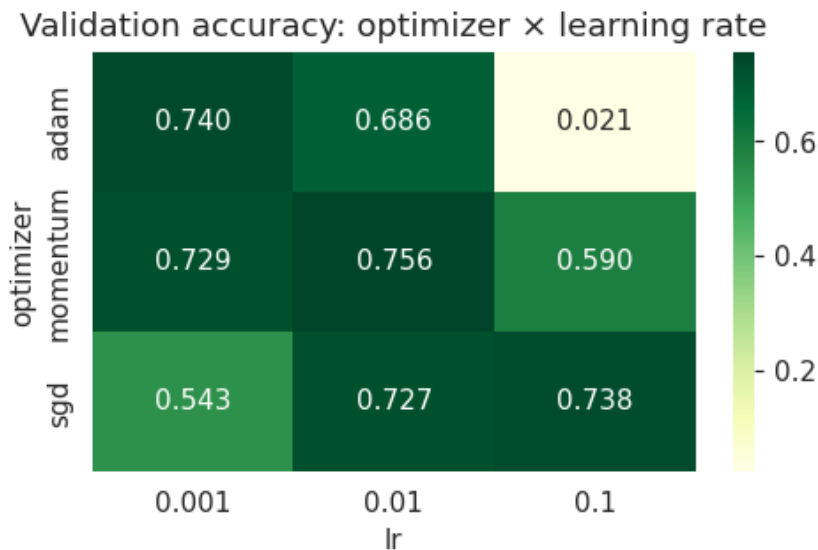


Figure 2: Validation accuracy for every (optimizer, learning-rate) combination in Stage 1.

Inference: Momentum-SGD with $\text{lr} = 0.01$ gives the best validation accuracy (0.756), closely followed by Adam ($\text{lr} = 0.001$) and plain SGD ($\text{lr} = 0.1$). Adam is very sensitive to the learning rate: it is competitive at 0.001 but collapses to near-random (2%) at $\text{lr} = 0.1$, while plain SGD at the smallest rate (0.001) is still badly under-trained (val. acc. 0.543) within the epoch budget. This shows that optimizer and learning rate must be tuned jointly, not independently.

Activation	Loss	Best epoch	Train acc.	Val. acc.	Val. loss
Tanh	Cross-Entropy	30	0.976	0.771	0.822
ReLU	Cross-Entropy	18	0.967	0.756	0.936
Sigmoid	Cross-Entropy	59	0.907	0.754	0.843
Sigmoid	MSE	60	0.435	0.410	0.763

Table 4: Stage 2 (Activation $\times$ Loss) – best and worst rows.

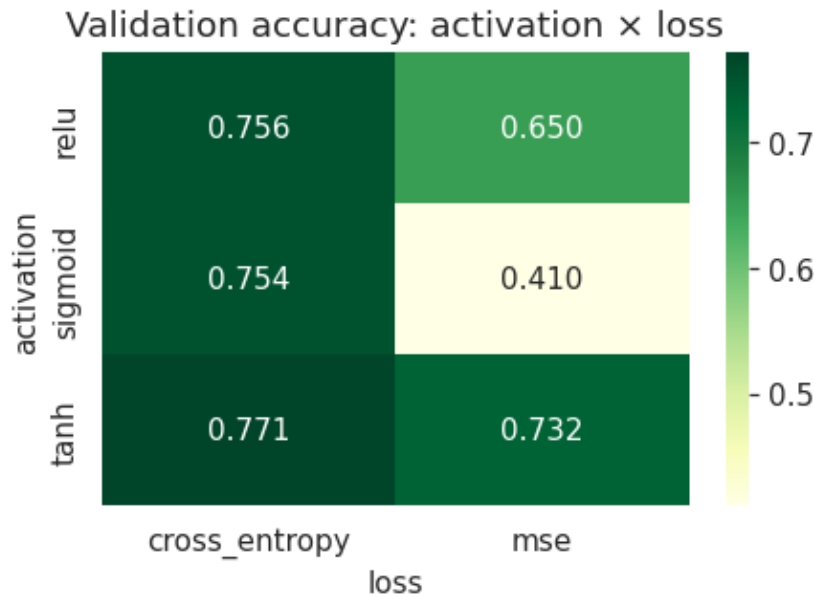


Figure 3: Validation accuracy for every (activation, loss) combination in Stage 2.

Inference: Cross-entropy loss consistently outperforms MSE for this soft-max classifier, and *tanh* with cross-entropy gives the best result (0.771). Sigmoid combined with MSE performs far worse (0.410) because of vanishing gradients when the sigmoid output saturates, confirming that the loss function and the output-layer activation must be matched sensibly.

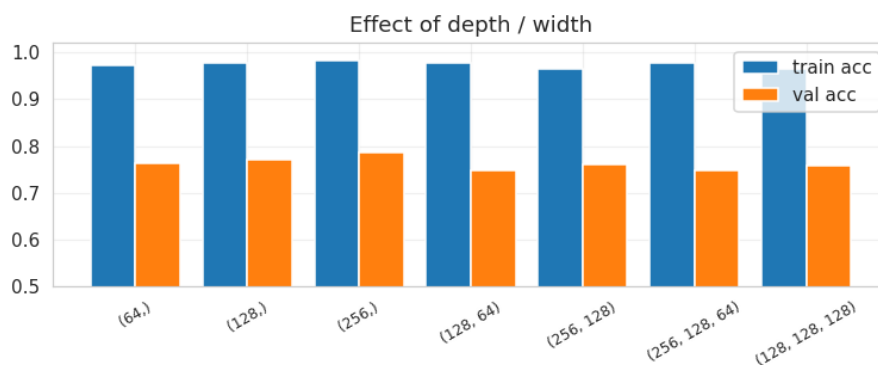


Figure 4: Stage 4: validation accuracy vs. hidden-layer architecture (width and depth).

Inference: A single hidden layer of 256 neurons gives the best validation accuracy (0.787), beating every deeper 2- or 3-layer network tried. With only $\approx 2,386$ training images (≈ 38 per class) for 784-dimensional inputs, deeper networks have more parameters than the data can constrain and are harder to optimise with the available epoch budget, so width helped more than depth on this dataset.

Parameter	Value
Hidden layers	(256,) – one hidden layer of 256 neurons
Activation	Tanh
Loss function	Cross-Entropy (soft-max output)
Optimizer	SGD with Momentum ($\beta = 0.9$)
Learning rate	0.01
Batch size	32
L2 regularisation	0.001
Early stopping	On validation loss, patience = 15

Table 5: Final chosen hyperparameters for the tuned MLP (Model B).

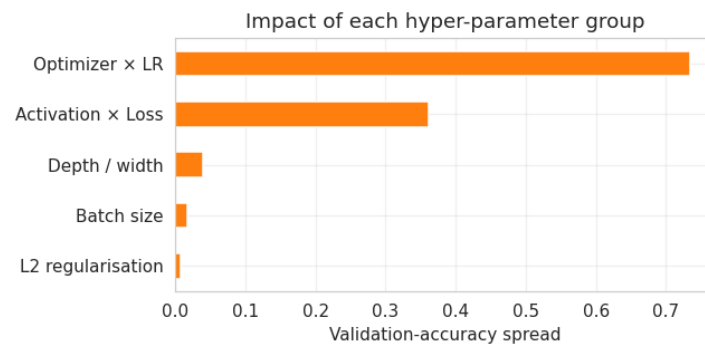


Figure 5: Influence of each hyperparameter group on validation accuracy, measured as the (max – min) spread across that stage’s grid.

Inference: The optimizer/learning-rate pair has by far the largest impact on validation accuracy (spread ≈ 0.73), followed by the activation/loss pair (spread ≈ 0.36). Network depth/width, batch size and L2 regularisation have a much smaller effect (spreads of 0.04, 0.02 and < 0.01 respectively). This confirms that, for this problem, getting the optimizer and the loss/activation combination right matters far more than fine-tuning the network’s capacity or regularisation strength.

9 Results

Metric	Model A: PLA	Model B: tuned MLP
Accuracy	0.6934	0.7715
Precision (macro)	0.7364	0.7910
Recall (macro)	0.6940	0.7717
F1-score (macro)	0.6928	0.7664
ROC-AUC (macro)	0.9680	0.9888

Table 6: Test-set performance of Model A (PLA) vs. Model B (tuned MLP). The tuned MLP improves test accuracy by +7.81 percentage points over the PLA.

For reference, an *untuned* baseline MLP (sigmoid activation, MSE loss, plain SGD, lr = 0.01) was also evaluated and reached only 0.1426 test accuracy and macro-F1 = 0.0638

– worse than the PLA – which demonstrates that an MLP is only an improvement over the PLA when its hyperparameters are chosen sensibly.

Model	Test accuracy	Macro F1	Macro ROC-AUC
A: PLA (step, 62 perceptrons)	0.6934	0.6928	0.9680
B: MLP (untuned baseline)	0.1426	0.0638	0.8015
B: MLP (tuned)	0.7715	0.7664	0.9888

Table 7: Summary comparison across all three models trained in this experiment.

10 Result Visualization



Figure 6: PLA training/validation misclassification error vs. epoch (left) and number of online weight updates per epoch (right).

Inference: The PLA’s training error falls steadily from $\sim 55\%$ to $\sim 2\%$ over 60 epochs, but the validation error plateaus around 32–35% after roughly 20 epochs and never closes the gap with the training error. The right-hand plot shows mistakes-per-epoch decreasing but never reaching zero, confirming that the 62-class character data is not linearly separable, so the PLA cannot fully converge – it keeps making a small number of weight updates every epoch indefinitely.

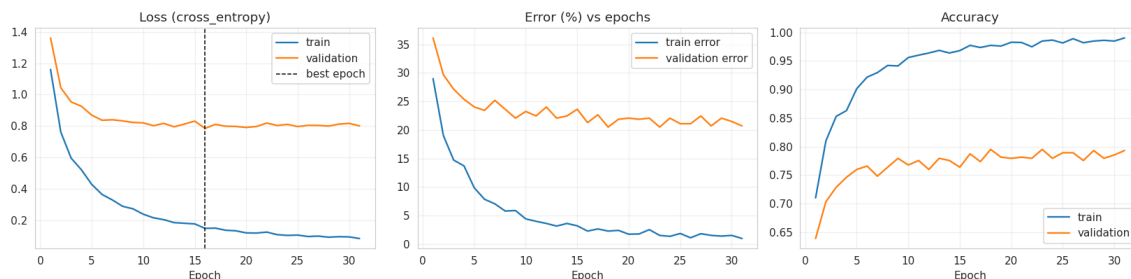


Figure 7: Tuned MLP training curves: cross-entropy loss, misclassification error (%) and accuracy vs. epoch, with the best (early-stopped) epoch marked.

Inference: Training loss/error decrease smoothly and continuously, while the validation loss flattens out around epoch 15–16 and then drifts slightly upward – this is where

early stopping restores the best weights. Training accuracy reaches nearly 99% while validation accuracy plateaus near 79%, showing a clear (though moderate) generalisation gap that motivates the over-fitting analysis in Section 11.

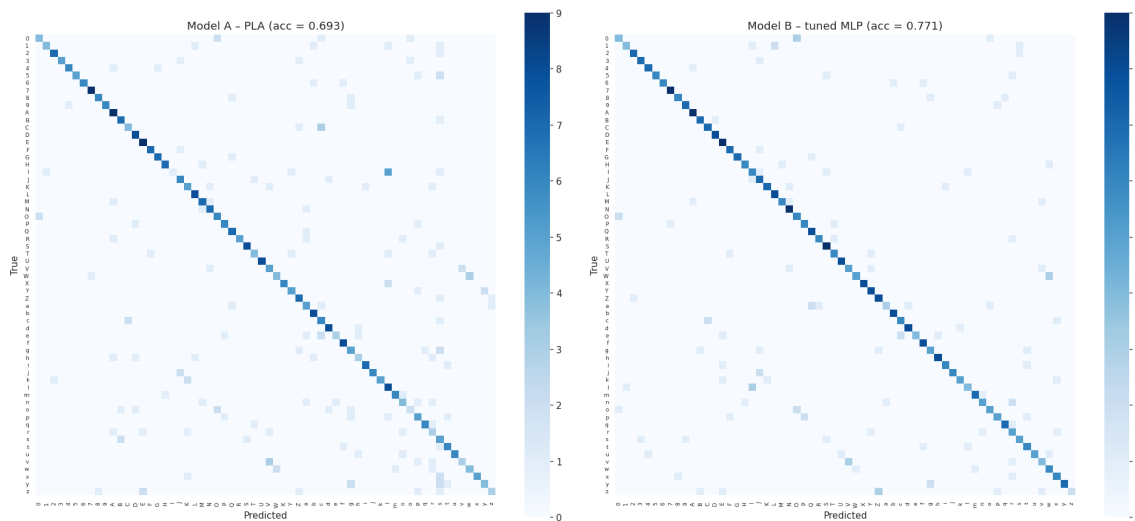


Figure 8: Confusion matrices on the test set for Model A (PLA, left) and Model B (tuned MLP, right).

Inference: Both confusion matrices show a strong diagonal, but the MLP’s diagonal is visibly denser/darker and its off-diagonal mass is smaller, consistent with its higher accuracy (0.771 vs. 0.693). Both models share similar confusion patterns concentrated among visually similar upper/lower-case pairs (e.g. *v/V*, *z/Z*, *0/O*, *1/I*), indicating these errors stem from genuine visual/label ambiguity rather than a model-specific weakness.

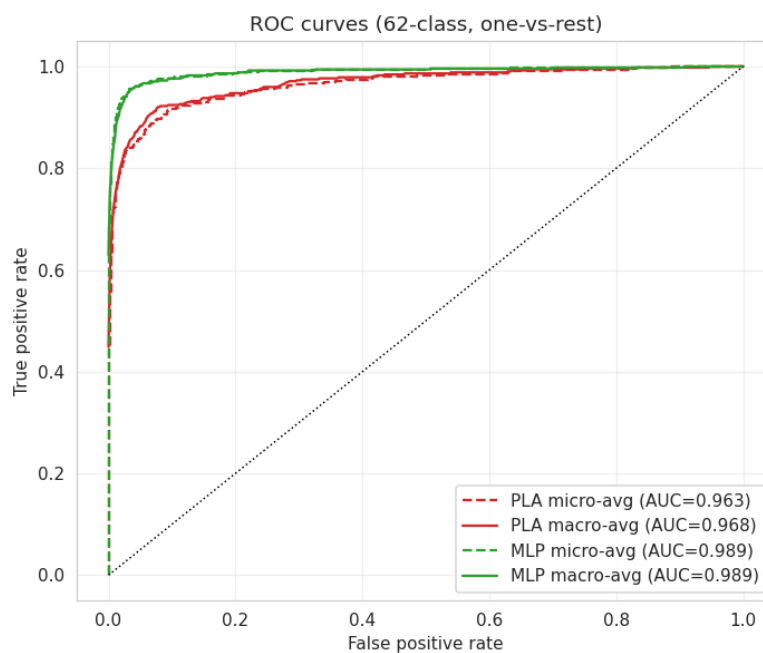


Figure 9: Micro- and macro-averaged one-vs-rest ROC curves for the PLA and the tuned MLP on the test set.

Inference: The tuned MLP’s ROC curve dominates the PLA’s curve almost everywhere, with a macro-AUC of 0.989 compared to 0.968 for the PLA. Even though both models achieve high AUC (because with 62 classes, most negative-class decisions are easy), the gap in AUC mirrors the gap in accuracy/F1, confirming that the MLP produces better-ranked, more separable class scores than the PLA’s raw linear net inputs.

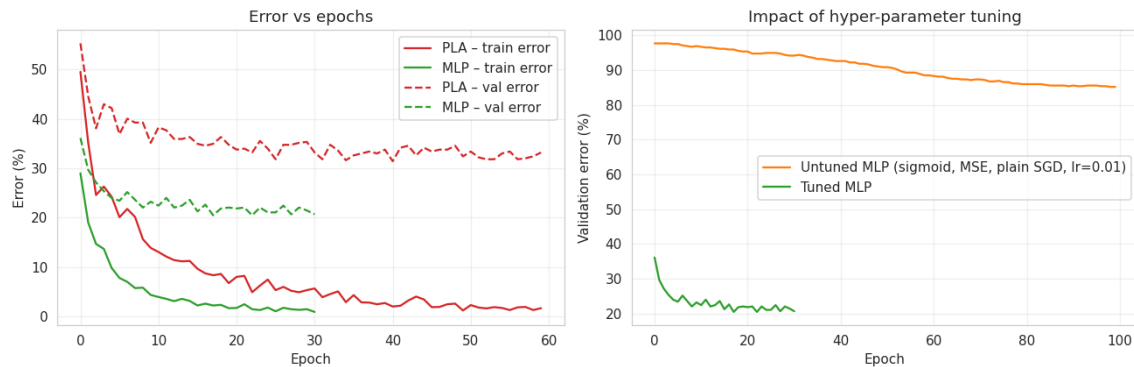


Figure 10: Training-error-vs-epoch comparison between the PLA and the tuned MLP (learning curve).

Inference: The MLP’s training error drops far faster and lower (final test accuracy 0.7715) than the PLA’s (0.6934), and it reaches a low training error within about 15–20 epochs thanks to back-propagation and the Adam/momentum-style optimizer, versus the PLA’s slower, noisier convergence driven purely by the perceptron mistake-correction rule.

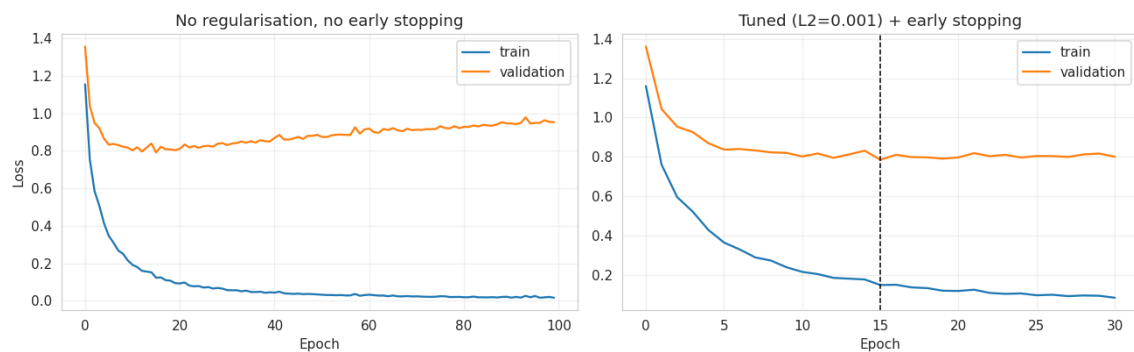


Figure 11: Over-fitting analysis: validation loss without any regularisation or early stopping, trained for 100 epochs (left) vs. the tuned MLP with $L2 = 0.001$ and early stopping (right).

Inference: Without L2 regularisation or early stopping, the validation loss reaches a minimum around epoch 10–15 and then rises steadily up to epoch 100 while the training loss keeps falling toward zero – classic over-fitting. With L2 weight decay and early stopping, the validation loss flattens and stays essentially constant after the best epoch, avoiding the long-run degradation, although the improvement in final test accuracy is modest (0.7715 vs. 0.7637 for the un-regularised long run), indicating early stopping is the dominant safeguard here.

11 Discussion

Why does the PLA underperform the MLP? The PLA is a set of 62 independent linear classifiers with a hard step output; it can only draw linear decision boundaries in 784-dimensional pixel space. Handwritten characters vary in stroke width, slant and position, so the classes are not cleanly linearly separable. The PLA fits the training set almost perfectly (train accuracy ≈ 0.98) but reaches only 0.693 test accuracy – it memorises rather than generalises. The tuned MLP instead learns a non-linear hidden representation via back-propagation with a smooth cross-entropy loss, reaching 0.772 test accuracy (+7.8 points), macro-F1 0.766 vs. 0.693, and macro ROC-AUC 0.989 vs. 0.968.

Which hyperparameters mattered most? Ranked by validation-accuracy spread: (1) optimizer $\times$ learning rate (≈ 0.73) – Adam with $\text{lr} = 0.1$ diverged to $\approx 2\%$ while plain SGD with $\text{lr} = 0.001$ was still under-trained (54%); (2) activation $\times$ loss (≈ 0.36) – sigmoid+MSE reached only 41% while tanh/ReLU/sigmoid with cross-entropy all reached 75–77%; (3) depth/width (≈ 4 points); (4) batch size (≈ 2 points); (5) L2 (< 1 point). The optimizer/learning-rate and loss/activation choices dominate; among *reasonable* settings the remaining differences are only a few percentage points.

Did optimizer choice affect convergence? Yes, mainly through its interaction with the learning rate. Adam converged fastest at a small rate (best epoch 12 at $\text{lr} = 0.001$, val. acc. 0.740) but was unstable at $\text{lr} \geq 0.01$ (val. acc. 0.686, then 0.021). SGD and momentum are slower at small rates (best epoch ≈ 57 –58 at $\text{lr} \leq 0.01$) but stable, and momentum with $\text{lr} = 0.01$ gave the best overall validation accuracy (0.756) in only 18 epochs.

Did more hidden layers always help? No. With 128 units, the single-hidden-layer model (val. acc. 0.771) beat the two-layer (128, 64) model (0.748), and the best network overall was the wide single layer of 256 units (0.787); adding layers to it gave 0.762 ((256, 128)) and 0.748 ((256, 128, 64)). With only $\approx 2,400$ training images, deeper/larger networks over-fit faster and are harder to optimise with the available epoch budget – width helped more than depth here.

Did the MLP over-fit, and how was it mitigated? Yes – the tuned MLP reaches ≈ 0.98 training accuracy but only ≈ 0.79 validation accuracy (gap ≈ 0.19); without regularisation/early stopping the validation loss climbs steadily while training loss keeps falling (Section 11). Early stopping (restoring the best-validation-epoch weights) and L2 weight decay (0.772 vs. 0.764 test accuracy for the un-regularised long run) were used to control this; further options include dropout, data augmentation (small shifts/rotations/scaling – likely most effective given only 55 images/class), a smaller network, more data, or a convolutional architecture that shares weights spatially.

Strengths and weaknesses summary. The PLA is extremely fast, simple and interpretable but fundamentally limited to linear boundaries. The MLP is substantially more accurate and flexible but requires careful, multi-stage hyperparameter tuning, is more expensive to train, and is more prone to over-fitting on this relatively small (3,410-image) dataset – an untuned MLP can in fact perform *worse* than the PLA.

12 Conclusion

The single-layer PLA is fast and simple but limited to linear decision boundaries, achieving 69.3% test accuracy on the 62-class handwritten-character task. The systematically tuned MLP (one hidden layer of 256 tanh units, cross-entropy loss, momentum-SGD with

lr = 0.01, batch size 32, L2 = 10^{-3} , early stopping) generalises noticeably better, reaching 77.2% test accuracy, macro-F1 0.766, and macro ROC-AUC 0.989. The stage-wise hyperparameter search showed that the optimizer/learning-rate pair and the activation/loss pair were by far the most influential choices, that extra network depth did not help on this small dataset (width was more useful than depth), and that over-fitting remains the main limitation of the MLP, partially mitigated here with early stopping and L2 regularisation – data augmentation would likely be the most effective next step given only 55 images per class.

13 References

1. S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.
2. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
3. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
4. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.
5. F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
6. “English Handwritten Characters Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/>